Getting the flight table into Notion

For whoever builds and runs the Notion side. It answers one question first, because the answer decides everything else.

Can Notion fetch this by itself?

Plain Notion cannot. Database automations in Notion trigger on things that happen *inside* Notion — a page created, a property changed, a date reached. There is no native automation that says "every five minutes, fetch this URL and write the rows into a database". The GitHub connector Notion ships syncs *issues and pull requests*, not files in a repository, so it is no help here either.

That leaves three honest routes. They differ in who owns the clock.

Route	Who runs the timer	Delay	Cost	Code
1. GitHub Actions *(recommended)*	GitHub, triggered by the app's own commit	seconds	none	a script, supplied below
2. Notion Worker (sync)	Notion, on a schedule	15 min at best	Business plan	a worker, deployed by CLI
3. Make, Zapier, n8n, API Connector	the service	1–15 min	subscription	none

Route 1 is the one to take, and it is not a workaround. The app already commits to the repository on every entry. That commit can start a workflow directly, so nothing polls anything: the entry is in Notion a few seconds after the pilot pressed the button. Route 2 and 3 both add a waiting room in front of a door that is already open.

Route 2 is worth knowing about because it is first-party. Notion Workers, introduced at Notion's 2026 developer day, run small pieces of code on Notion's servers and can pull external data on a schedule — every 15 minutes, hourly, daily. They need the Business plan and the Notion CLI. For a flight being followed live, a quarter of an hour is too coarse; for an archive built after landing it would do.

What the app writes

One file per flight, in the data repository, always the complete table:

```
data/<flight-id>.json          every row, sorted by ts_utc
data/<flight-id>.csv           the same rows as CSV
data/<flight-id>.deleted.json  ids the crew removed
data/_setup.json               the crew's own configuration, not flight data
data/_flights.json             an index of closed flights
```

Each row carries a stable `id` (a UUID). **Upsert on that id. Never append.** The file holds the whole table, not a delta; an integration that appends will double the table on its second run.

Route 1, step by step

1. A Notion database

Create a database in the workspace. Give it a **Name** column of type *title* — the script puts the row's `id` there, and that is what makes the upsert work. Then add whichever of these you want; the script skips any column that does not exist and says so in the run log rather than failing:

Column	Type	Column	Type
Name	title	Direction	select
Flight	text	Station	text
Callsign	text	Frequency	text
Type	select	Squawk	text
Time	date	Message	text
Reporter	text	PIC	text
Device	text	Resting	text
Position	text	Battery	number
Longitude	text	FuelCell	select
Place	text	Methanol	number
Altitude	number	Solar	select
Track	number	CrewO2	select
Speed	number	O2Litres	number
Vario	number	O2Percent	number
Action	select	Note	text
Medium	select	Edited	text
Delta	number	Ballast	number
Sand	number	Water	number

Keep the Edited column. The script uses it to tell an unchanged row from a corrected one, and that is what keeps a run from rewriting two hundred pages every time.

2. A Notion integration

1. notion.com/my-integrations → **New integration**, internal, in the right workspace
2. Capabilities: **Read content**, **Update content**, **Insert content**
3. Copy the token — it begins with `ntn_`
4. Open the database in Notion → ■ → **Connections** → add the integration

Without step 4 the integration cannot see the database, and every call answers 404. It is the step people miss.

3. The database id

From the database URL:

```
https://www.notion.so/workspace/8f2c1a04b7e34f6d9a5c0e1b2d3f4a5b?v=...
                               ■■■■■■■■■■ the 32 characters you need ■■■■■■■■■■
```

4. The files, in the data repository

```
notion-sync/sync.mjs                  the script
.github/workflows/notion-sync.yml     the workflow (from notion-sync/notion-sync.yml)
```

The workflow file has to sit at `.github/workflows/`, the script wherever the workflow points — `notion-sync/` in the supplied version.

5. Two secrets

Repository → Settings → Secrets and variables → Actions → **New repository secret**

```
NOTION_TOKEN      ntn_...
NOTION_DATABASE   8f2c1a04b7e34f6d9a5c0e1b2d3f4a5b
```

6. Run it once by hand

Actions → *Notion sync* → **Run workflow**. The log ends with a line like

```
rows 128 · created 128 · updated 0 · archived 0
```

From then on it runs on its own: every commit the app makes starts it, and a quarter-hourly schedule catches anything a failed run missed.

What the script does, and what it costs

It reads the flight files, drops the rows listed in the tombstone file, fetches the pages already in the database once, and then writes only what changed — a new row is created, a corrected row is patched, an unchanged row costs no request at all. A row the crew deleted is **archived** rather than destroyed, so nothing is lost by accident.

Requests are spaced by about a third of a second, which sits inside Notion's roughly three per second. A flight of two hundred entries costs two hundred writes spread over the flight, not in one burst, because each entry starts its own run.

GitHub Actions minutes: public repositories are free; a private repository has 2,000 minutes a month on the Free plan. A run takes about twenty seconds, so a flight with two hundred entries uses a little over an hour of that budget. If that becomes tight, remove the `push` trigger and leave the schedule — the table is complete in every run, so a fifteen-minute rhythm loses nothing but immediacy.

Route 3, if no code is wanted at all

Make, Zapier, n8n and Notion's own API Connector can all do this without a repository. The shape is the same in each: a scheduled HTTP request, then a Notion upsert.

```
GET https://api.github.com/repos/<owner>/gb_flight_data/contents/data/<flight-id>.json
Authorization: Bearer <read-only GitHub token>
Accept: application/vnd.github.raw
X-GitHub-Api-Version: 2022-11-28
```

`Accept: application/vnd.github.raw` returns the file itself; without it GitHub sends an envelope with the content base64-encoded inside. Iterate the array, upsert on `id`, and poll no faster than the service allows. Send

`If-None-Match` with the previous ETag if the tool supports it — an unchanged file then answers `304` and costs nothing against the GitHub rate limit.

When rows do not arrive

Symptom	Cause
Every call answers 404	The integration was never added to the database under *Connections*.
The workflow does not start on a commit	It is not on the default branch, or the path filter does not match the folder in use.
Rows double on the second run	The integration is appending instead of upserting on `id`.
A deleted row reappears	`<flight>.deleted.json` is not being read.
Everything is rewritten every run	The **Edited** column is missing, so the script cannot tell an unchanged row.
429 from Notion	Requests are going out too fast; the supplied script already paces itself and retries.